ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ (UET)

BÁO CÁO MÔN HỌC

SMART ATTENDANCE SYSTEM

Hệ Thống Điểm Danh Thông Minh

Phiên bản: `v1.0.3`

Ngày tạo: `Ngày 28 tháng 5 năm 2026`

Giảng viên hướng dẫn:

Đào Việt Anh Lương Sơn Bá Trịnh Ngọc Huỳnh

Nhóm sinh viên thực hiện (Nhóm 9):

1. 24022454 Lê Việt Thành (Trưởng nhóm - DevOps Lead)
2. 24022406 Nguyễn Nhật Minh (Backend Developer)
3. 24022364 Tạ Duy Khánh (AI/ML Engineer)
4. 24022394 Nguyễn Đức Mạnh (Frontend Developer)

Hà Nội, Tháng 05/2026

Mục lục

Danh mục từ viết tắt **5**

1 Tổng quan dự án **6**
1.1 Bối cảnh và bài toán 6
1.2 Mục tiêu 6
1.3 Phạm vi và đối tượng sử dụng 6

2 Yêu cầu và công nghệ **8**
2.1 Yêu cầu chức năng 8
2.2 Yêu cầu phi chức năng 8
2.3 Công nghệ sử dụng 9

3 Phân tích và thiết kế **10**
3.1 Use Case tổng thể 10
3.1.1 Use Case chi tiết: Đăng ký sinh viên 12
3.1.2 Use Case chi tiết: Điểm danh 13
3.1.3 Use Case chi tiết: Quản trị hệ thống 15
3.2 Kiến trúc tổng thể 17
3.3 Luồng xử lý 19
3.4 Thiết kế cơ sở dữ liệu 21

4 Hiện thực hóa hệ thống **22**
4.1 Cấu trúc mã nguồn 22
4.2 Backend API 22
4.3 Luồng điểm danh và xử lý đa phiên 23
4.4 AI Worker và Vector DB 23
4.5 Frontend 24
4.6 Giao diện demo 24

5 Triển khai và vận hành **26**
5.1 Docker Compose và dịch vụ 26
5.2 Docker Volumes 26
5.3 Cấu hình môi trường 27

5.4 Sao lưu dữ liệu 27

6 Kiểm thử và đánh giá **28**
6.1 Kịch bản kiểm thử 28
6.2 Đánh giá hiệu năng (tham khảo) 28
6.3 Hạn chế 28

7 Tính năng nâng cao **29**
7.1 Các hạng mục đã triển khai 29

8 Phân công nhiệm vụ **30**

9 Kết luận và hướng phát triển **31**
9.1 Kết luận 31
9.2 Hướng phát triển 31

Danh sách hình vẽ

3.1 Sơ đồ Use Case tổng thể hệ thống 11
3.2 Sơ đồ Use Case chi tiết – Phân hệ Đăng ký sinh viên 12
3.3 Sơ đồ Use Case chi tiết – Phân hệ Điểm danh 14
3.4 Sơ đồ Use Case chi tiết – Phân hệ Quản trị 16
3.5 Sơ đồ thành phần (Component Diagram) 17
3.6 Sơ đồ triển khai (Deployment Diagram) với Docker 18
3.7 Sơ đồ tuần tự cho luồng đăng ký/điểm danh 19
3.8 Sơ đồ hoạt động cho luồng điểm danh 1-1 20
3.9 Sơ đồ ERD cơ sở dữ liệu 21

4.1 Các màn hình quản trị chính của hệ thống 24
4.2 Màn hình đăng ký điểm danh cho sinh viên 24
4.3 Màn hình quản lý sinh viên (Admin) 25
4.4 Các màn hình chức năng phía sinh viên 25

Danh sách bảng

1.1 Đối tượng sử dụng và phạm vi nghiệp vụ 7

2.1 Bảng công nghệ chính của dự án 9

3.1 Mô tả Use Case phân hệ Đăng ký 13
3.2 Mô tả Use Case phân hệ Điểm danh 15
3.3 Mô tả Use Case phân hệ Quản trị 17
3.4 Mô tả các thực thể chính 21

5.1 Danh sách dịch vụ chính trong Docker Compose 26
5.2 Danh sách Docker Volumes chính 26
5.3 Các biến môi trường quan trọng 27

6.1 Các kịch bản kiểm thử chức năng chính 28
6.2 Bảng phân rã độ trễ cho luồng điểm danh 28

8.1 Thông tin thành viên và tỉ lệ đóng góp 30
8.2 Chi tiết nhiệm vụ của từng thành viên 30

Danh mục từ viết tắt

Từ viết tắt	**Ý nghĩa**
AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
CI/CD	Continuous Integration / Continuous Deployment
CNN	Convolutional Neural Network (Mạng nơ-ron tích chập)
DB	Database (Cơ sở dữ liệu)
ERD	Entity-Relationship Diagram (Sơ đồ thực thể liên kết)
REST	Representational State Transfer
HNSW	Hierarchical Navigable Small World
JWT	JSON Web Token

Chương 1

Tổng quan dự án

1.1 Bối cảnh và bài toán

Chuyển đổi số trong giáo dục đòi hỏi các hệ thống quản lý học vụ phải chính xác, minh bạch và giảm tối đa thao tác thủ công. Các phương pháp điểm danh truyền thống (gọi tên, thẻ từ, vân tay) đều gặp hạn chế về thời gian, chi phí và khả năng gian lận. Dự án **Smart Attendance System** hướng tới một quy trình điểm danh tự động hóa bằng nhận diện khuôn mặt, bảo đảm xác minh đúng danh tính theo mã sinh viên.

1.2 Mục tiêu

- **Xác minh 1-1 theo mã sinh viên**: hạn chế nhầm lẫn và loại bỏ điểm danh hộ.
- **Tối ưu thời gian**: giảm thời gian điểm danh xuống mức giây cho mỗi lượt.
- **Kiến trúc mở rộng**: tách luồng AI ra khỏi API bằng Celery/Redis để xử lý đồng thời nhiều yêu cầu.
- **Quản trị rõ ràng**: hỗ trợ duyệt sinh viên, duyệt lịch học, sửa trạng thái điểm danh, xuất báo cáo.

1.3 Phạm vi và đối tượng sử dụng

Hệ thống xử lý hai luồng nghiệp vụ chính: *Đăng ký hồ sơ sinh viên* và *Điểm danh Quản trị.*

Đối tượng	Nghiệp vụ cốt lõi	Quyền hạn tương tác
Sinh viên	Đăng ký khuôn mặt, đăng ký lịch học	Gửi ảnh/video đăng ký, cập nhật lịch học, điểm danh check-in/check-out.
Giảng viên/Admin	Duyệt sinh viên, duyệt lịch học, quản trị điểm danh	Xem thống kê, nhật ký, sửa trạng thái, xuất CSV.

Bảng 1.1: Đối tượng sử dụng và phạm vi nghiệp vụ

Chương 2

Yêu cầu và công nghệ

2.1 Yêu cầu chức năng

- Đăng ký sinh viên bằng ảnh hoặc video, lưu vector khuôn mặt.
- Quản lý lịch học theo ca, có cơ chế duyệt/huỷ.
- Điểm danh 1-1 theo mã sinh viên, hỗ trợ check-in/check-out đa phiên.
- Cronjob 23:59 tự động quét vắng và đánh dấu quên check-out theo lịch học.
- Phát hiện đi muộn theo ngưỡng thời gian, cảnh báo và lưu log.
- Dashboard thống kê, nhật ký điểm danh, thông báo và xuất báo cáo CSV.
- Quản trị người dùng và quyền truy cập thông qua JWT.

2.2 Yêu cầu phi chức năng

- Độ trễ xử lý thấp (mục tiêu dưới 2 giây mỗi lượt điểm danh trong môi trường demo).
- Khả năng mở rộng bằng cách tăng số lượng worker.
- Dữ liệu ảnh và vector lưu tách rời, bảo đảm an toàn và dễ sao lưu.
- Dễ triển khai với Docker Compose, cấu hình bằng biến môi trường.

2.3 Công nghệ sử dụng

Tầng	**Công nghệ**	**Vai trò**
Frontend	Vue 3 + Vite	Giao diện điểm danh và dashboard quản trị.
Backend	FastAPI + SQLAlchemy	API xử lý nghiệp vụ, auth, thống kê.
AI Worker	Celery + DeepFace	Trích xuất vector, so khớp khuôn mặt.
Vector DB	Qdrant	Lưu và truy vấn vector theo student_id.
RDBMS	MySQL 8.0	Lưu thông tin sinh viên, lịch học, log điểm danh.
Cache/Queue	Redis	Hàng đợi Celery và cache.
Object Storage	MinIO	Lưu ảnh gốc và ảnh điểm danh.
Gateway	Nginx	Reverse proxy cho frontend và API.
Deploy	Docker Compose	Đóng gói, triển khai nhanh toàn hệ thống.

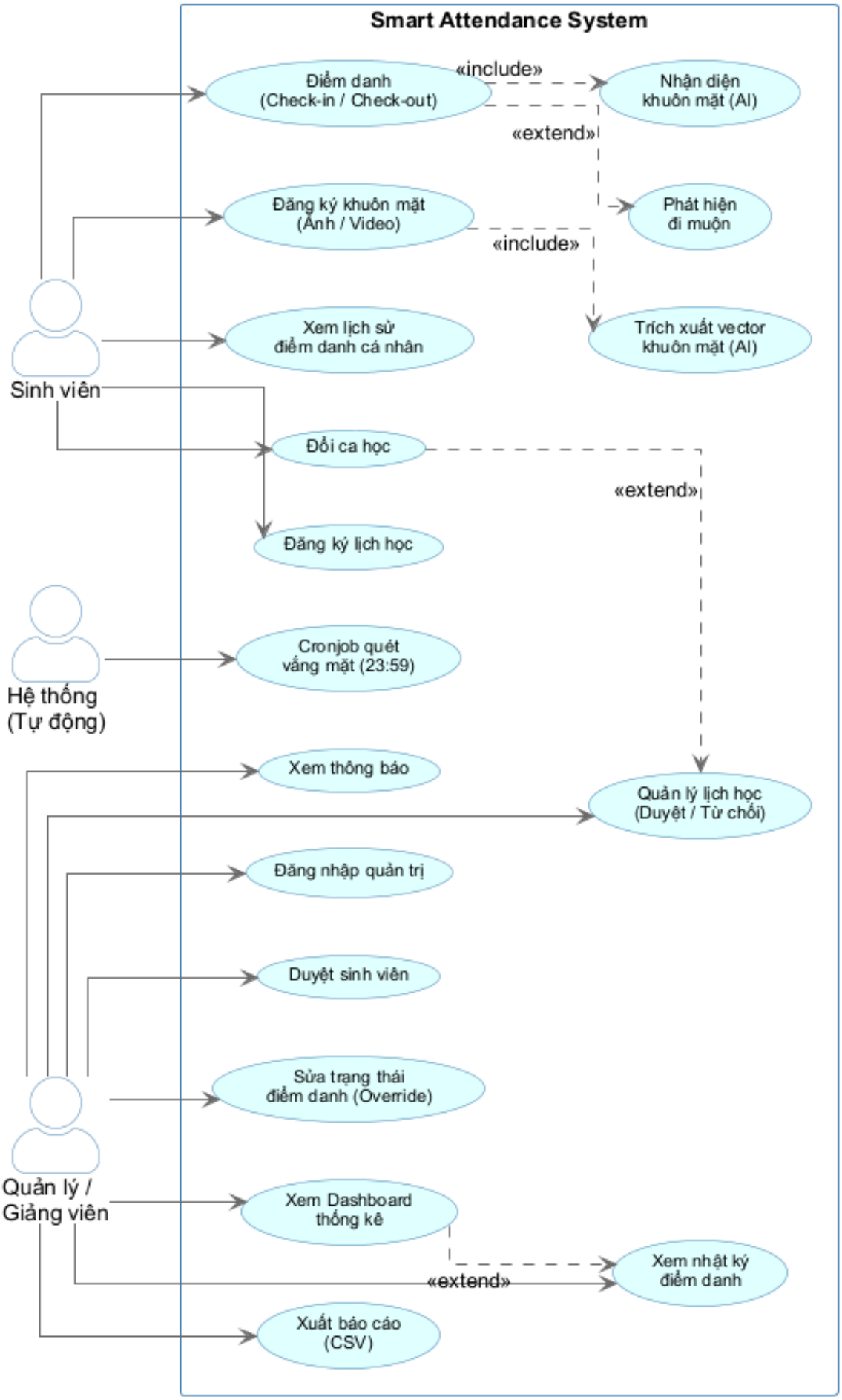
Bảng 2.1: Bảng công nghệ chính của dự án

Chương 3

Phân tích và thiết kế

3.1 Use Case tổng thể

Sơ đồ Use Case tổng thể (Hình 3.1) thể hiện toàn bộ chức năng của hệ thống theo ba nhóm tác nhân chính: Sinh viên, Quản lý/Giảng viên, và Hệ thống tự động.



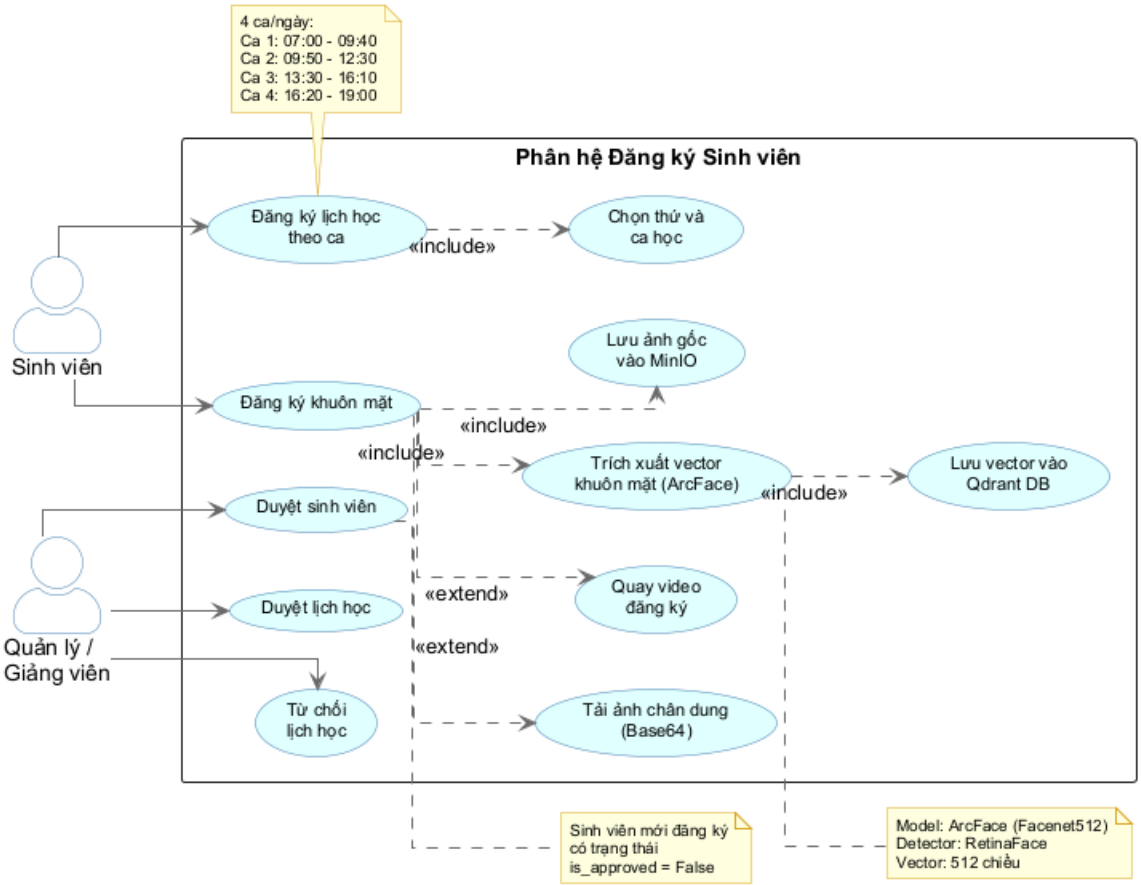
Hình 3.1: Sơ đồ Use Case tổng thể hệ thống

Hệ thống bao gồm các nhóm chức năng sau:

- **Sinh viên**: Đăng ký khuôn mặt (ảnh/video), đăng ký lịch học, đổi ca học, điểm danh check-in/check-out, xem lịch sử cá nhân.
- **Quản lý/Giảng viên**: Đăng nhập quản trị, duyệt sinh viên, quản lý lịch học (duyệt/từ chối), sửa trạng thái điểm danh, xem dashboard thống kê, xem nhật ký, xuất CSV, xem thông báo.
- **Hệ thống**: Trích xuất vector khuôn mặt (AI), nhận diện khuôn mặt (AI), phát hiện đi muộn, Cronjob quét vắng mặt tự động lúc 23:59.

3.1.1 Use Case chi tiết: Đăng ký sinh viên

Sơ đồ dưới đây (Hình 3.2) mô tả chi tiết quy trình đăng ký sinh viên mới, bao gồm đăng ký khuôn mặt (bằng ảnh hoặc video), đăng ký lịch học theo ca, và quy trình xét duyệt bởi giảng viên.



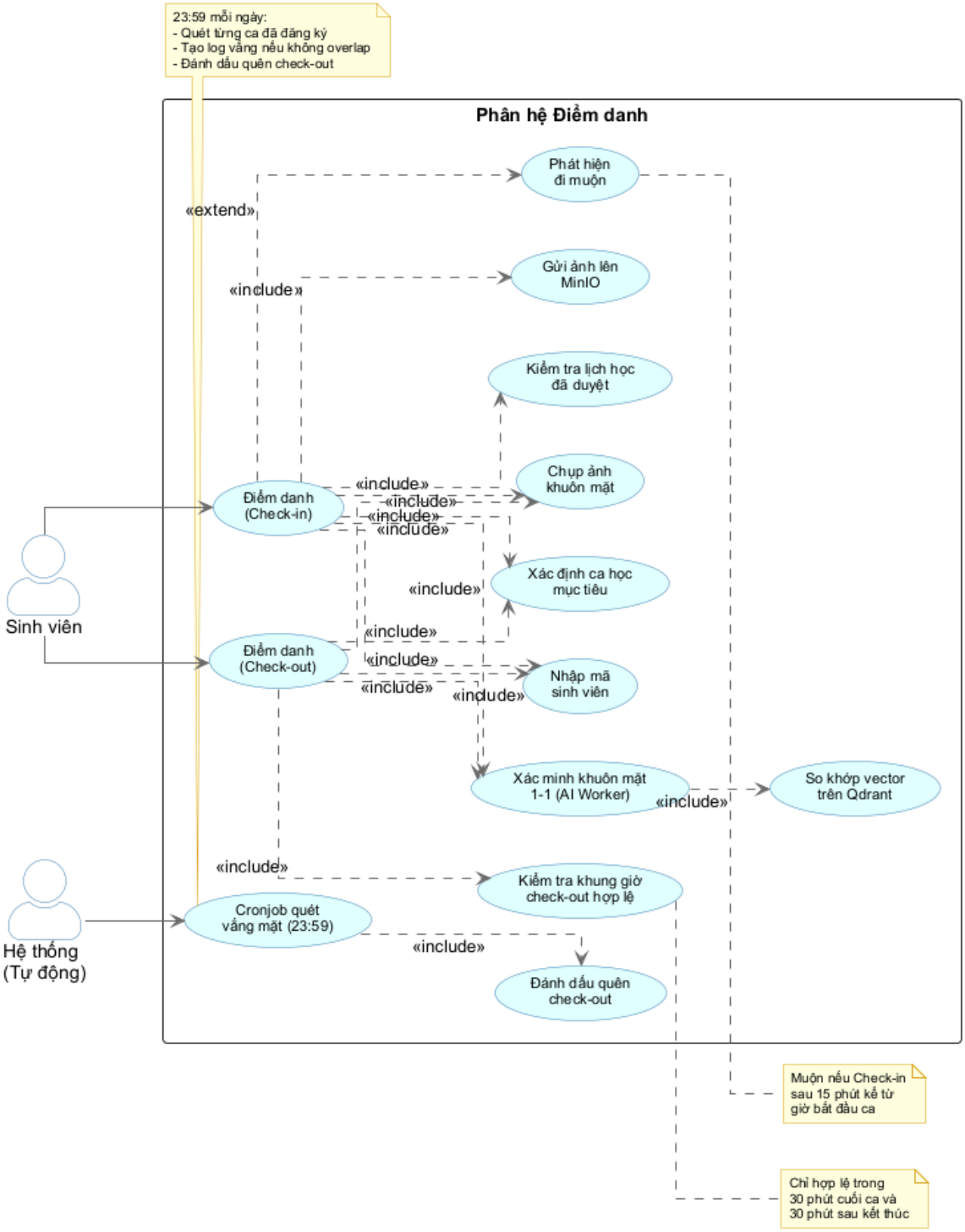
Hình 3.2: Sơ đồ Use Case chi tiết – Phân hệ Đăng ký sinh viên

Use Case	Mô tả
Đăng ký khuôn mặt	Sinh viên tải ảnh (base64) hoặc quay video. Hệ thống trích xuất vector 512 chiều bằng ArcFace, lưu vào Qdrant và ảnh gốc vào MinIO.
Đăng ký lịch học	Sinh viên chọn thứ và ca học (4 ca/ngày). Lịch ở trạng thái chờ duyệt.
Duyệt sinh viên	Giảng viên duyệt tài khoản mới (is_approved = True).
Duyệt/Từ chối lịch học	Giảng viên phê duyệt hoặc từ chối yêu cầu lịch. Lịch cũ bị vô hiệu hóa khi có lịch mới được duyệt.

Bảng 3.1: Mô tả Use Case phân hệ Đăng ký

3.1.2 Use Case chi tiết: Điểm danh

Sơ đồ dưới đây (Hình 3.3) mô tả chi tiết quy trình điểm danh đa phiên, bao gồm check-in, check-out, nhận diện khuôn mặt AI, phát hiện đi muộn và xử lý tự động cuối ngày.



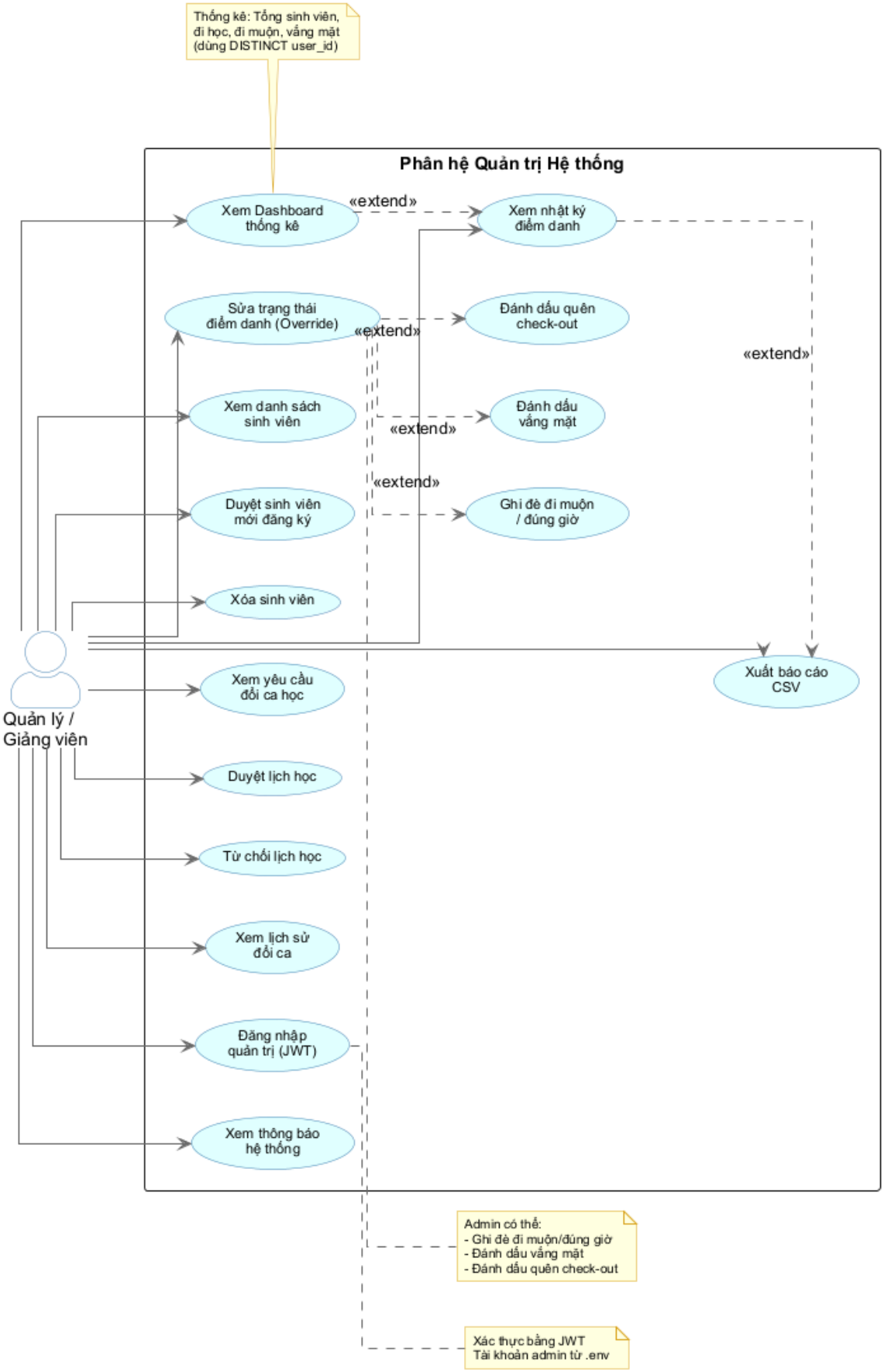
Hình 3.3: Sơ đồ Use Case chi tiết – Phân hệ Điểm danh

Use Case	Mô tả
Check-in	Sinh viên nhập MSSV, chụp ảnh. Hệ thống kiểm tra lịch học, gửi ảnh qua Celery Worker xác minh 1-1 với Qdrant. Nếu khớp, ghi log check-in và xác định ca mục tiêu.
Check-out	Tương tự check-in nhưng kiểm tra thêm khung giờ hợp lệ (30 phút cuối ca + 30 phút sau kết thúc).
Phát hiện đi muộn	Tự động đánh dấu nếu check-in sau 15 phút so với giờ bắt đầu ca.
Cronjob quét vắng	Lúc 23:59, quét từng ca đã đăng ký. Tạo log vắng nếu không tìm thấy phiên giao cắt. Đánh dấu quên check-out cho phiên chưa đóng.

Bảng 3.2: Mô tả Use Case phân hệ Điểm danh

3.1.3 Use Case chi tiết: Quản trị hệ thống

Sơ đồ dưới đây (Hình 3.4) mô tả chi tiết các chức năng quản trị dành cho giảng viên/admin, bao gồm quản lý sinh viên, quản lý lịch học, override điểm danh, dashboard và báo cáo.

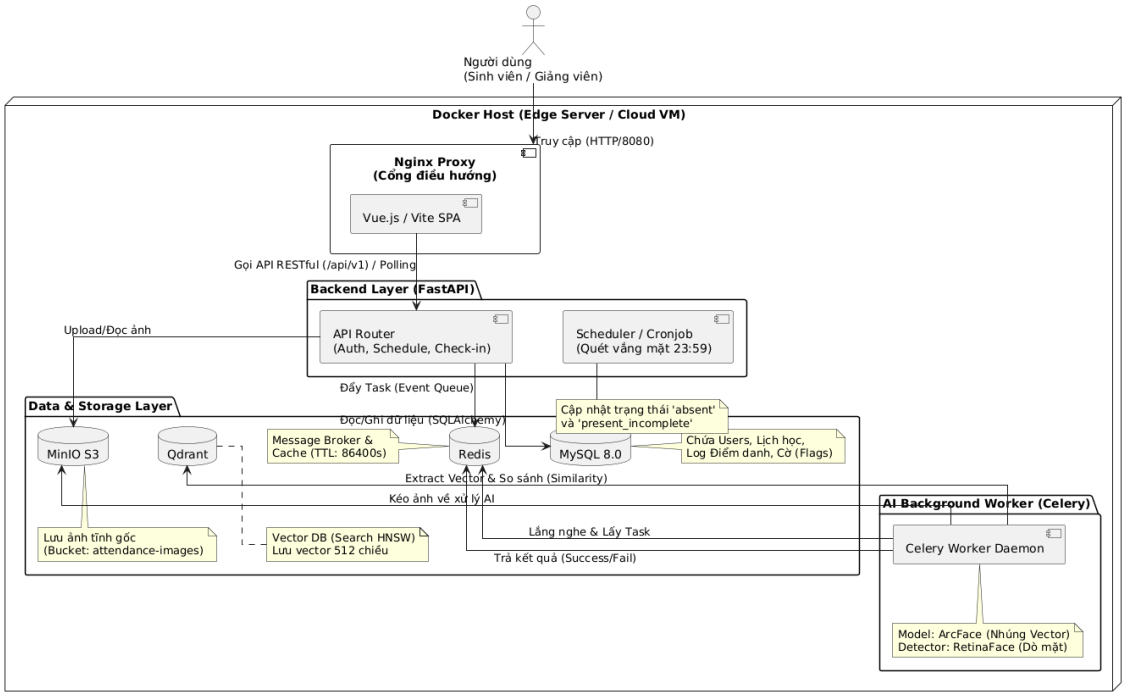


Hình 3.4: Sơ đồ Use Case chi tiết – Phân hệ Quản trị

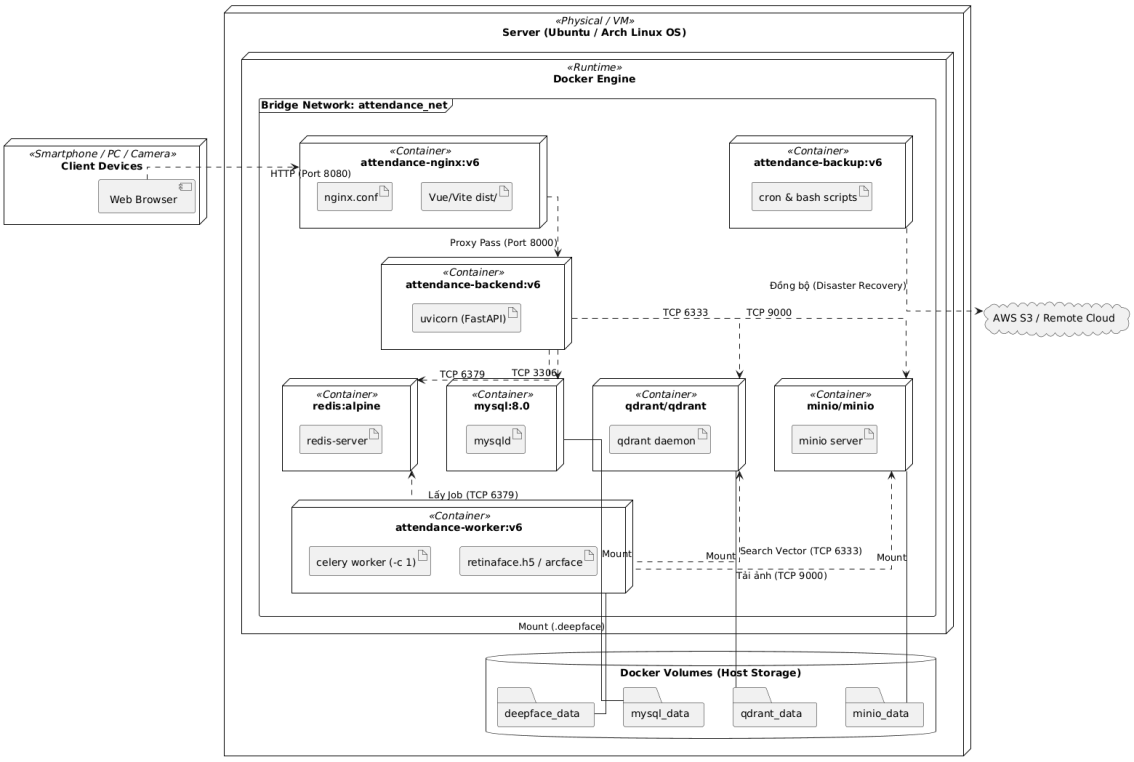
Use Case	Mô tả
Đăng nhập quản trị	Xác thực bằng JWT. Tài khoản admin cấu hình từ biến môi trường.
Duyệt/Xóa sinh viên	Phê duyệt tài khoản sinh viên mới hoặc xóa sinh viên khỏi hệ thống.
Quản lý lịch học	Duyệt/từ chối yêu cầu đổi ca. Xem lịch sử đổi ca của sinh viên.
Override điểm danh	Ghi đè trạng thái: đi muộn/đúng giờ, đánh dấu vắng, đánh dấu quên check-out.
Dashboard thống kê	Xem tổng quan: tổng sinh viên, đi học, đi muộn, vắng mặt (dùng DISTINCT user_id).
Xuất CSV	Xuất nhật ký điểm danh chi tiết ra file CSV để phục vụ báo cáo.

Bảng 3.3: Mô tả Use Case phân hệ Quản trị

3.2 Kiến trúc tổng thể

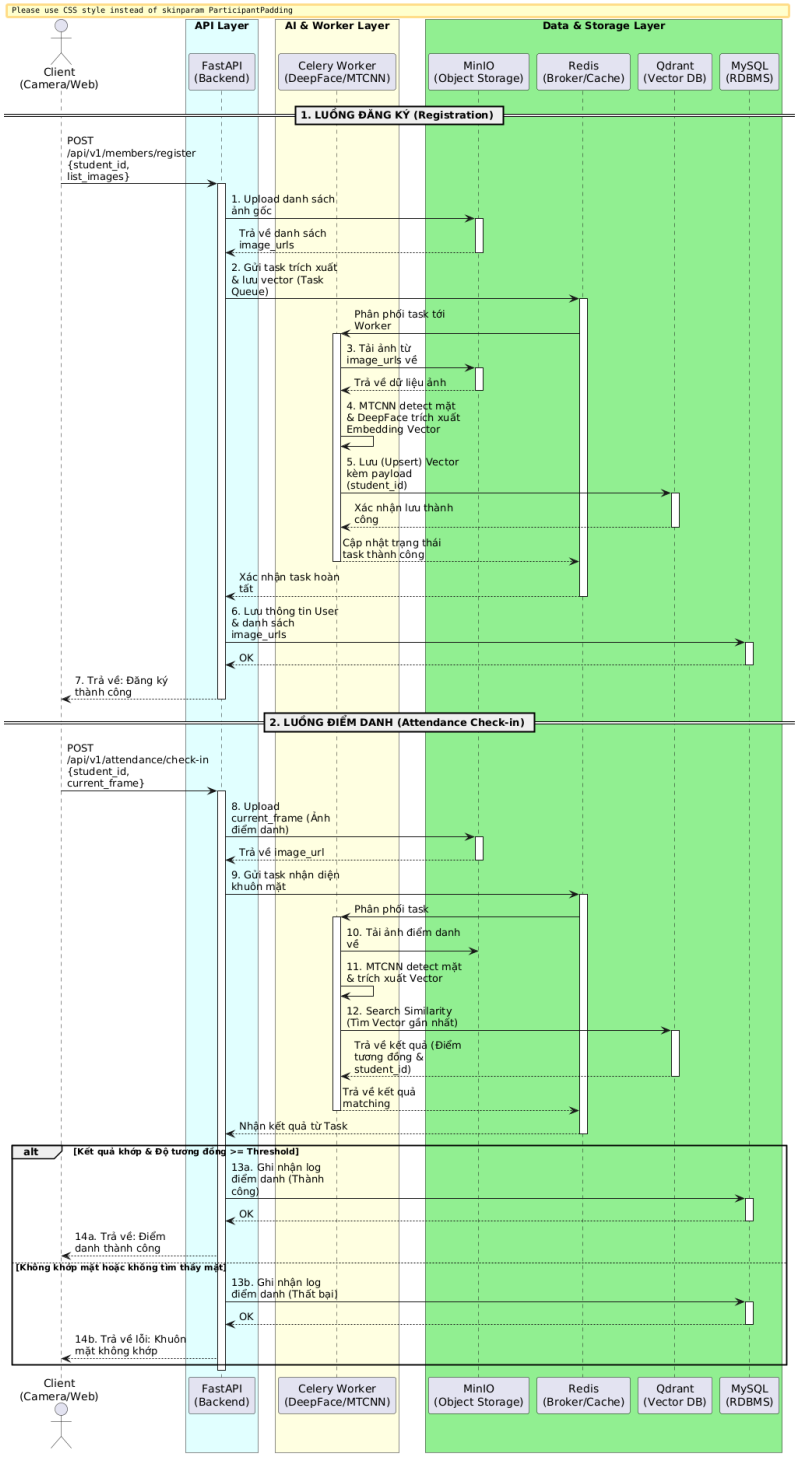


Hình 3.5: Sơ đồ thành phần (Component Diagram)



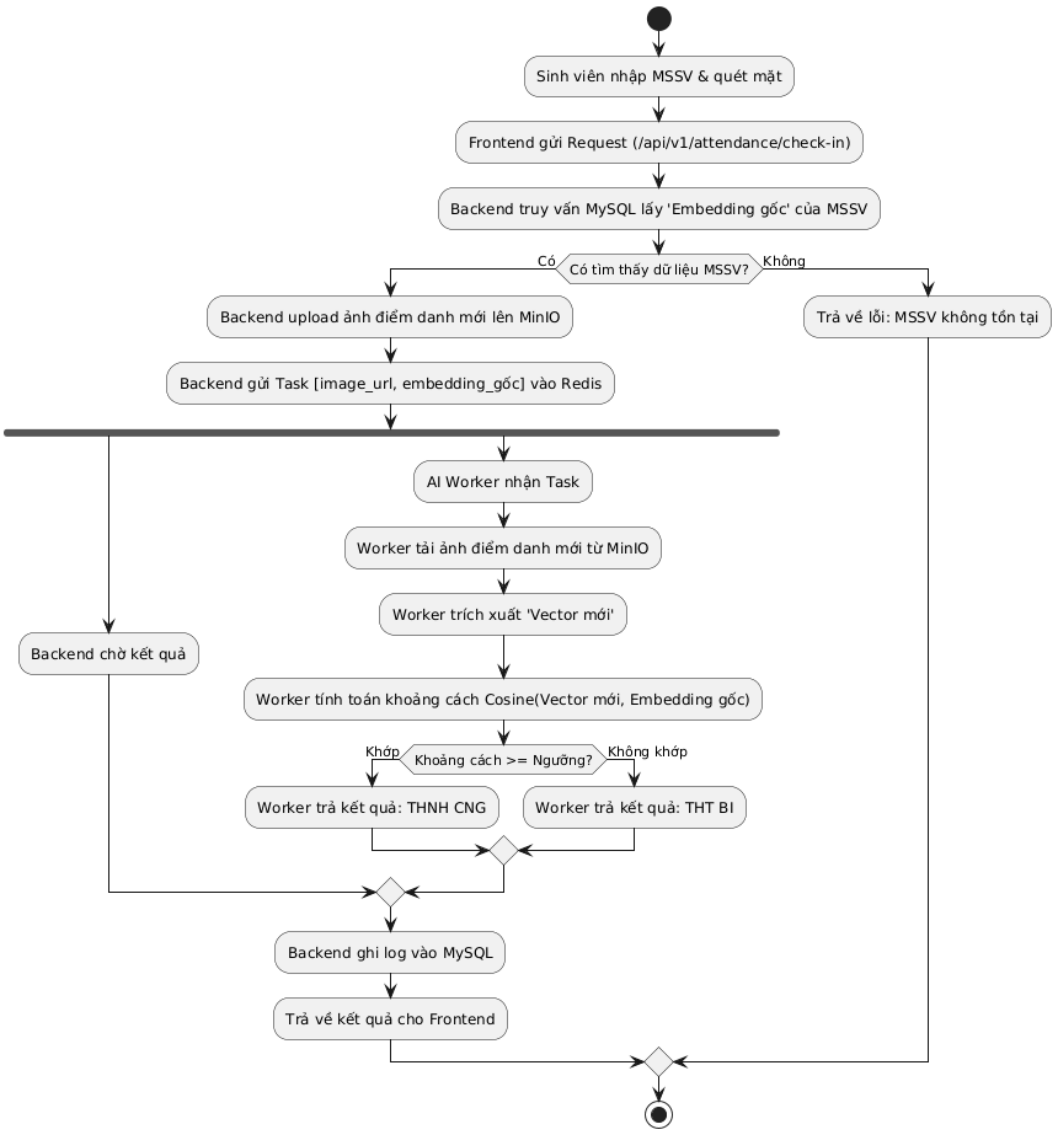
Hình 3.6: Sơ đồ triển khai (Deployment Diagram) với Docker

3.3 Luồng xử lý



Hình 3.7: Sơ đồ tuần tự cho luồng đăng ký/điểm danh

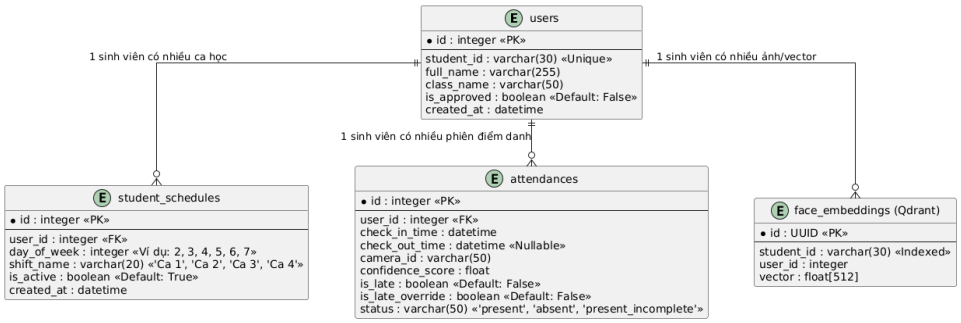
Hình 3.7 mô tả chuỗi xử lý chính: frontend gửi yêu cầu, backend kiểm tra dữ liệu và tải ảnh lên MinIO, sau đó đẩy tác vụ vào Redis để worker trích xuất vector, so khớp với Qdrant và trả kết quả về backend ghi log.



Hình 3.8: Sơ đồ hoạt động cho luồng điểm danh 1-1

Hình 3.8 thể hiện luồng xác minh 1-1 theo mã sinh viên: nếu không có mã hoặc không tồn tại dữ liệu vector thì trả lỗi sớm; nếu có, worker trích xuất vector, tính độ tương đồng với ngưỡng, trả về kết quả khớp/không khớp để backend ghi nhận.

3.4 Thiết kế cơ sở dữ liệu



Hình 3.9: Sơ đồ ERD cơ sở dữ liệu

Bảng/Collection	**Mô tả**
users	Thông tin sinh viên, trạng thái duyệt.
student_schedules	Lịch học theo ngày/ca, trạng thái duyệt và lịch sử đổi ca.
attendances	Log điểm danh (check-in/check-out), trạng thái, độ tin cậy.
face_embeddings (Qdrant)	Vector khuôn mặt 512 chiều, payload student_id.

Bảng 3.4: Mô tả các thực thể chính

Chương 4

Hiện thực hóa hệ thống

4.1 Cấu trúc mã nguồn

```
.
|-- docker-compose.yml
|-- docker-compose-network.yml
|-- backend/
|-- worker/
|-- frontend/
|-- nginx/
|-- backup/
|-- uml/
|-- report/
```

Listing 4.1: Tổ chức thư mục dự án theo chuẩn Microservices

4.2 Backend API

Hệ thống API được tổ chức theo chuẩn REST, nhóm theo các module: Auth, Members, Schedule, Attendance, Dashboard.

Endpoint	**Chức năng**
`/api/v1/auth/login`	Đăng nhập admin, trả JWT.
`/api/v1/members/register`	Đăng ký sinh viên bằng ảnh base64.
`/api/v1/members/register-video`	Đăng ký sinh viên bằng video.
`/api/v1/members`	Danh sách sinh viên.
`/api/v1/members/{student_id}/approve`	Duyệt sinh viên.
`/api/v1/members/{student_id}(DELETE)`	Xóa sinh viên.
`/api/v1/schedule/{student_id}`	Xem/cập nhật lịch học.

`/api/v1/schedule/{student_id}/approve`	Duyệt lịch học.
`/api/v1/schedule/{student_id}/reject`	Từ chối lịch học.
`/api/v1/schedule/{student_id}/history`	Lịch sử đổi ca.
`/api/v1/attendance/check-in`	Điểm danh (check-in hoặc check-out).
`/api/v1/attendance/{attendance_id}/override`	Sửa trạng thái điểm danh.
`/api/v1/dashboard/statistics`	Thống kê điểm danh trong ngày.
`/api/v1/dashboard/recent-activities`	Hoạt động gần đây.
`/api/v1/dashboard/attendance-logs`	Nhật ký điểm danh phân trang.
`/api/v1/dashboard/notifications`	Đếm thông báo.
`/api/v1/dashboard/notifications/list`	Danh sách thông báo.
`/api/v1/dashboard/export-attendance`	Xuất CSV nhật ký điểm danh.
`/health`	Kiểm tra trạng thái dịch vụ.

4.3 Luồng điểm danh và xử lý đa phiên

Luồng điểm danh sử dụng cơ chế xác minh 1-1 theo mã sinh viên. Backend kiểm tra lịch học đã duyệt, gửi ảnh lên MinIO, đẩy tác vụ tới Celery Worker, nhận kết quả khớp để ghi log. Một ngày có thể có nhiều phiên học, vì vậy hệ thống hỗ trợ *multi-session*:

- **Check-in**: được phép từ 30 phút trước ca đến khi ca kết thúc.
- **Check-out**: chỉ hợp lệ trong 30 phút cuối ca và 30 phút sau khi ca kết thúc.
- **Đi muộn**: nếu check-in sau 15 phút kể từ thời điểm bắt đầu ca.

Cuối ngày (23:59), scheduler tự động quét các ca học đã đăng ký: nếu không có phiên điểm danh giao cắt thì tạo log vắng mặt, đồng thời đánh dấu các phiên thiếu check-out là *present_incomplete*.

4.4 AI Worker và Vector DB

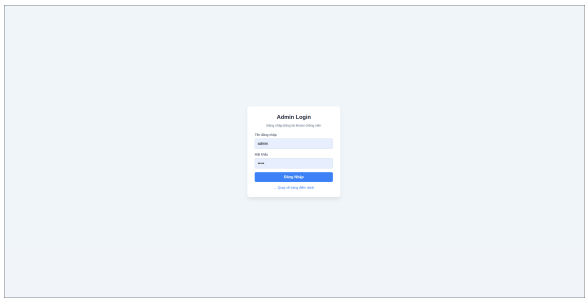
Worker sử dụng DeepFace (ArcFace) để trích xuất vector 512 chiều, kết hợp bộ dò tìm mặt RetinaFace (có thể cấu hình qua biến môi trường). Vector được so khớp trên Qdrant với bộ lọc `student_id`, tránh quét toàn bộ dữ liệu và giữ độ chính xác cao.

4.5 Frontend

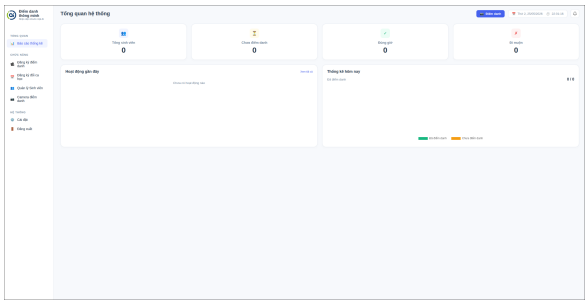
Giao diện Vue 3 gồm hai vai trò: Sinh viên (check-in/check-out, nhập mã sinh viên) và Admin (dashboard thống kê, duyệt sinh viên, duyệt lịch học, xem log, xuất CSV).

4.6 Giao diện demo

Các màn hình dưới đây minh họa luồng thao tác chính của hệ thống từ đăng nhập, theo dõi thống kê đến điểm danh.

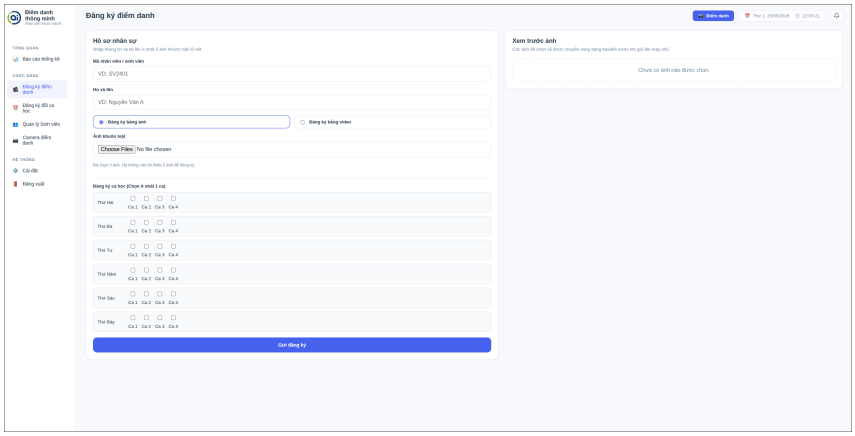


(a) Đăng nhập Admin

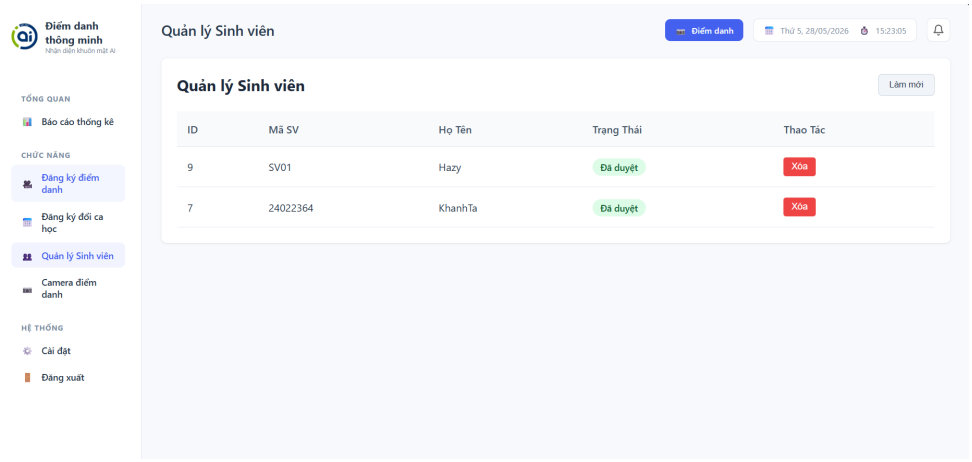


(b) Dashboard thống kê

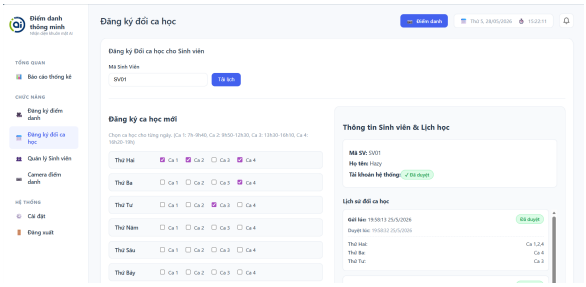
Hình 4.1: Các màn hình quản trị chính của hệ thống



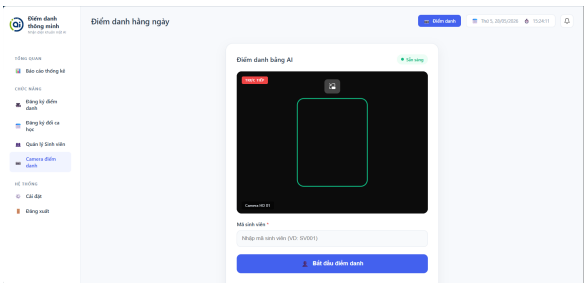
Hình 4.2: Màn hình đăng ký điểm danh cho sinh viên



Hình 4.3: Màn hình quản lý sinh viên (Admin)



(a) Giao diện đổi ca học



(b) Giao diện điểm danh sinh viên

Hình 4.4: Các màn hình chức năng phía sinh viên

Chương 5

Triển khai và vận hành

5.1 Docker Compose và dịch vụ

Hệ thống đóng gói 8 container độc lập:

Service	**Chức năng**	**Cổng**
nginx	Reverse proxy, phục vụ frontend	8080
backend	API FastAPI	8000 (internal)
worker	Celery Worker AI	-
mysql	CSDL quan hệ	3307
redis	Queue/Cache	6379
qdrant	Vector DB	6333
minio	Object Storage	9000/9001
backup	Sao lưu định kỳ	-

Bảng 5.1: Danh sách dịch vụ chính trong Docker Compose

5.2 Docker Volumes

Các volume được sử dụng để lưu dữ liệu bền vững và cache model:

Volume	**Mục đích**
mysql_data	Lưu dữ liệu MySQL.
qdrant_data	Lưu vector embeddings trong Qdrant.
minio_data	Lưu object storage (ảnh điểm danh).
deepface_data	Cache model DeepFace trong worker.

Bảng 5.2: Danh sách Docker Volumes chính

5.3 Cấu hình môi trường

Các biến môi trường quan trọng nằm trong file `.env`:

Biến	**Ý nghĩa**
`DATABASE_URL/MYSQL_*`	Thông tin kết nối MySQL.
`QDRANT_URL,QDRANT_COLLECTION`	Kết nối và tên collection Qdrant.
`MINIO_ENDPOINT,MINIO_ACCESS_KEY, MINIO_SECRET_KEY`	Kết nối MinIO.
`FACE_MATCH_THRESHOLD`	Ngưỡng khớp khuôn mặt.
`DEEPFACE_MODEL_NAME,FACE_DETECTOR_ BACKEND`	Cấu hình mô hình và detector.
`ADMIN_USERNAME,ADMIN_PASSWORD,JWT_ SECRET`	Tài khoản quản trị và khóa JWT.
`TIME_LATE`	Ngưỡng xác định đi muộn.

Bảng 5.3: Các biến môi trường quan trọng

5.4 Sao lưu dữ liệu

Container `backup` thực hiện sao lưu định kỳ MySQL, snapshot Qdrant và đồng bộ ảnh từ MinIO về thư mục `local_backups/`. Chính sách retention mặc định giữ 7 ngày gần nhất để tối ưu dung lượng.

Chương 6

Kiểm thử và đánh giá

6.1 Kịch bản kiểm thử

Tình huống	**Kỳ vọng**
Đúng người, đúng student_id	Điểm danh thành công.
Sai người, đúng student_id	Thất bại, trả về lý do không khớp.
Đúng người, sai student_id	Thất bại, trả về điểm thấp.
Thiếu student_id	Trả lỗi 400.
student_id chưa đăng ký vector	Trả lỗi 404.
Check-out trước giờ cho phép	Trả lỗi không hợp lệ.

Bảng 6.1: Các kịch bản kiểm thử chức năng chính

6.2 Đánh giá hiệu năng (tham khảo)

Công đoạn xử lý	**Thời gian (ms)**	**Ghi chú**
FastAPI upload MinIO	~ 50	I/O nội bộ.
Celery + DeepFace inference	~ 800	Sau khi pre-warm model.
Qdrant 1-1 verification	~ 20	Lọc theo student_id.
Tổng end-to-end	< 1.5 giây	Môi trường demo nội bộ.

Bảng 6.2: Bảng phân rã độ trễ cho luồng điểm danh

6.3 Hạn chế

- Chưa tích hợp *liveness detection*, dễ bị spoof bằng ảnh tĩnh.
- Điều kiện ánh sáng kém hoặc che mặt có thể giảm độ chính xác.

Chương 7

Tính năng nâng cao

7.1 Các hạng mục đã triển khai

Nhóm đã hoàn thiện một số hạng mục nâng cao nhằm tăng tính sẵn sàng, khả năng triển khai và vận hành thực tế:

- **CI/CD**: thiết lập pipeline tự động build và đóng gói Docker image, đảm bảo quy trình phát hành nhất quán.
- **Triển khai Internet**: hệ thống được public qua domain: `https://attendance.thanhtech.cloud` với SSL và Nginx gateway.
- **Backup dữ liệu**: container backup định kỳ dump MySQL, snapshot Qdrant và mirror MinIO, giữ lịch sử 7 ngày.
- **Quản lý cấu hình model/pipeline**: tham số mô hình và ngưỡng nhận diện cấu hình qua `.env` (`DEEPFACE_MODEL_NAME`, `FACE_DETECTOR_BACKEND`, `FACE_MATCH_THRESHOLD`).
- **Cache và hiệu năng**: Redis làm broker/cache, TTL cho embedding giúp giảm tải truy vấn và ổn định độ trễ.

Chương 8

Phân công nhiệm vụ

Bảng dưới đây trình bày chi tiết phân công nhiệm vụ của từng thành viên trong nhóm và tỉ lệ đóng góp tương ứng.

STT	MSSV	Họ và tên	Vai trò chính	Tỉ lệ
1	24022454	Lê Việt Thành	Trưởng nhóm, DevOps	25%
2	24022406	Nguyễn Nhật Minh	Backend Developer	25%
3	24022364	Tạ Duy Khánh	AI/Worker Developer	25%
4	24022394	Nguyễn Đức Mạnh	Frontend Developer	25%

Bảng 8.1: Thông tin thành viên và tỉ lệ đóng góp

Thành viên	Nhiệm vụ chi tiết
Lê Việt Thành	Thiết kế kiến trúc hệ thống, cấu hình Docker Compose, thiết lập CI/CD pipeline, cấu hình Nginx reverse proxy, xây dựng Backup Service, triển khai hệ thống lên Internet (SSL + domain), viết tài liệu báo cáo.
Nguyễn Nhật Minh	Phát triển API FastAPI (Auth, Members, Schedule, Attendance, Dashboard), thiết kế database schema MySQL, xây dựng Scheduler/Cronjob quét vắng mặt, tích hợp xác thực JWT, xử lý logic điểm danh đa phiên.
Tạ Duy Khánh	Xây dựng Celery Worker, tích hợp DeepFace (ArcFace/RetinaFace), triển khai Qdrant Vector DB, phát triển pipeline trích xuất và so khớp vector khuôn mặt, cấu hình Redis cache cho embeddings.
Nguyễn Đức Mạnh	Phát triển giao diện Vue 3 SPA, xây dựng trang điểm danh sinh viên, dashboard quản trị, giao diện quản lý sinh viên và lịch học, thiết kế UI/UX, tích hợp API với frontend.

Bảng 8.2: Chi tiết nhiệm vụ của từng thành viên

Chương 9

Kết luận và hướng phát triển

9.1 Kết luận

Dự án đã xây dựng thành công một hệ thống điểm danh thông minh với xác minh 1-1, kiến trúc microservices và quy trình vận hành đầy đủ (đăng ký, duyệt, điểm danh, báo cáo). Cấu trúc tách rời giữa API và AI Worker giúp hệ thống ổn định khi xử lý đồng thời nhiều lượt điểm danh.

9.2 Hướng phát triển

- Bổ sung liveness detection để chống giả mạo.
- Tối ưu scale bằng cách tự động tăng/giảm số lượng worker.
- Nâng cấp báo cáo và dashboard theo thời gian thực (WebSocket).

— HẾT —